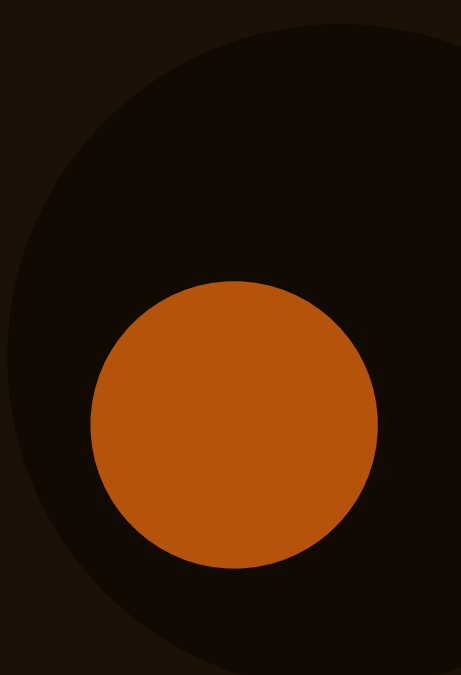




JAVA

Controle de Fluxo em Java

switch, for, while, Scanner e o debugger do IntelliJ

Apostila completa sobre laços, condições avançadas e leitura de dados do console em Java.

CONTEÚDO DA APOSTILA

- 01 Switch Tradicional e Avançado
- 02 O Laço for
- 03 while e do-while
- 04 Escopo de Variáveis Locais
- 05 Classes, Objetos e static
- 06 Scanner e Tratamento de Exceções
- 07 O Debugger do IntelliJ

JavaStudiesHub

Sumário

01

Switch Tradicional e Avançado

Do fall-through ao enhanced switch com yield

02

O Laço for

Estrutura, break, continue e aplicações práticas

03

while e do-while

A diferença fundamental entre os dois laços baseados em condição

04

Escopo de Variáveis Locais

Onde uma variável existe — e onde ela deixa de existir

05

Classes, Objetos e static

Conceitos essenciais para entender o Scanner

06

Scanner e Tratamento de Exceções

Lendo dados do console com segurança

07

O Debugger do IntelliJ

Inspecionando a execução do programa linha a linha

01

Switch Tradicional e Avançado

Do fall-through clássico ao enhanced switch expression com yield

Uma alternativa mais legível a vários if-else

O switch entra em cena quando você precisa comparar o valor de uma mesma variável contra várias constantes possíveis — nesse cenário, ele costuma ser mais legível (e, em certos casos, mais eficiente) do que uma longa cadeia de if-else. Os tipos aceitos pelo switch são byte, short, int, char, String e enums; ele não aceita long, float, double nem boolean.

```
// Tradicional (dois-pontos + break):
switch (categoriaProduto) {
    case 1: System.out.println("Hortifruti"); break;
    case 2: System.out.println("Laticínios"); break;
    default: System.out.println("Outros"); break;
}

// Enhanced switch EXPRESSION (Java 14+, forma recomendada):
String nomeSecao = switch (categoriaProduto) {
    case 1 -> "Hortifruti";
    case 2 -> "Laticínios";
    case 3 -> "Padaria";
    default -> "Categoria inválida";
};

// yield – quando o case precisa de mais de uma linha:
String resultado = switch (codigoDesconto) {
    case 1 -> "Sem desconto";
    case 2 -> {
        String msg = "Aplicando cupom...";
        System.out.println(msg);
        yield "10% de desconto"; // yield devolve o valor do bloco
    }
    default -> "Cupom inválido";
};
```

ATENÇÃO — Fall-through

Sem break, a execução do switch tradicional cai automaticamente para o próximo case — o chamado fall-through. Às vezes isso é proposital, para agrupar cases com o mesmo comportamento; outras vezes é um bug acidental. No enhanced switch com a seta ->, esse comportamento simplesmente não existe.

02

O Laço for

A estrutura de repetição mais usada quando o número de iterações já é conhecido

As três partes do for

O for é o laço preferido sempre que a quantidade de repetições pode ser calculada de antemão. Sua estrutura tem três partes separadas por ponto e vírgula: a inicialização (executada uma única vez, antes de tudo), a condição (verificada antes de cada volta — assim que ela se torna false, o laço para) e o incremento (executado ao final de cada volta).

```
// Estrutura básica:
for (int i = 0; i < 5; i++) {
    System.out.println("Item " + i + " do carrinho"); // 0, 1, 2, 3, 4
}

// break – encerra o laço imediatamente:
for (int i = 0; i < 100; i++) {
    if (i == 5) break;
    System.out.println("Conferindo item " + i); // imprime 0 a 4
}

// continue – pula direto para a próxima volta:
for (int i = 0; i <= 10; i++) {
    if (i % 2 != 0) continue; // pula os itens de índice ímpar
    System.out.println("Item " + i + " em promoção");
}
```

03

while e do-while

Dois laços baseados em condição — e o detalhe que os diferencia

A ordem em que a condição é verificada

Os dois laços repetem código enquanto uma condição for verdadeira, mas divergem no momento exato em que essa condição é checada. O `while` verifica antes de executar o bloco — se a condição já começar `false`, o corpo do laço pode nunca rodar (zero ou mais execuções). Já o `do-while` executa o bloco primeiro e só depois verifica a condição, garantindo pelo menos uma execução (uma ou mais vezes).

```
// while – condição verificada antes:
int caixa = 0;
while (caixa < 15) {
    caixa++;
    if (caixa <= 5) continue; // pula os 5 primeiros caixas
    if (caixa >= 10) break; // encerra ao chegar no caixa 10
    System.out.println("Caixa " + caixa + " aberto");
}

// do-while – roda ao menos uma vez (útil para menus):
int opcao;
do {
    System.out.println("1-Abrir caixa 2-Fechar caixa 0-Sair");
    opcao = scanner.nextInt();
    processarOpcaoCaixa(opcao);
} while (opcao != 0); // atenção ao ; obrigatório após o while!
```

REGRA

Prefira o `for` quando a quantidade de repetições já for conhecida. Prefira `while` ou `do-while` quando o número de voltas depender de uma condição que só se resolve durante a execução. Use especificamente o `do-while` quando o bloco precisar rodar pelo menos uma vez, como em menus e telas de validação de entrada.

04

Escopo de Variáveis Locais

Definindo onde, exatamente, uma variável pode ser usada

O bloco define o limite de vida da variável

Escopo é a região do código onde uma variável pode ser referenciada. Uma variável local só existe dentro do bloco em que foi declarada — e dentro de blocos aninhados dentro dele. Assim que a execução sai desse bloco, a variável deixa de existir; tentar usá-la fora dali gera erro de compilação.

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i); // i está em escopo aqui  
}  
// System.out.println(i); // ERRO: i está fora de escopo!  
  
// Diferença entre if e switch tradicional:  
// if: cada bloco {} tem escopo próprio  
if (totalCarrinho > 0) { int desconto = 10; }  
// else { desconto++; } // ERRO: desconto não existe fora daquele bloco  
  
// switch tradicional sem chaves: a variável pode vazar entre cases  
switch (categoria) {  
    case 1: int bonus = 5; // sem break – vaza para o próximo case!  
    case 2: bonus++; // funciona, mas é confuso  
}
```

05

Classes, Objetos e static

Os conceitos fundamentais para entender e usar o Scanner

Classe é o molde, objeto é a instância

Uma classe funciona como um molde — um tipo de dado customizado que descreve a estrutura de algo, mas ainda não é a coisa em si. Um objeto é a instância concreta desse molde, criada com a palavra-chave **new**. Campos marcados como **static** pertencem à classe como um todo e são compartilhados por todas as instâncias; campos de instância, por outro lado, pertencem individualmente a cada objeto criado.

```
// Criando um objeto Scanner para ler o código de barras digitado:
import java.util.Scanner;
Scanner leitor = new Scanner(System.in);

// Membros estáticos são acessados pelo nome da classe:
int max = Integer.MAX_VALUE; // campo static
double pi = Math.PI; // campo static
int abs = Math.abs(-42); // método static → 42
int codigo = Integer.parseInt("789"); // método static → 789
```


06

Scanner e Tratamento de Exceções

Lendo dados do console com segurança, usando try-catch

Capturando entrada do usuário

A classe Scanner, parte da biblioteca padrão do Java, permite ler dados digitados no console. Para capturar uma linha inteira de texto, incluindo espaços, o método adequado é `nextLine()`. Quando o texto lido precisa virar um número, entram em cena métodos estáticos como `Integer.parseInt()` ou `Double.parseDouble()` — e, se o texto não for um número válido, esses métodos lançam uma `NumberFormatException`.

```
import java.util.Scanner;

Scanner leitor = new Scanner(System.in);
double totalCompra = 0;
int itensLidos = 0;

while (itensLidos < 5) {
    System.out.print("Digite o preço do item #" + (itensLidos + 1) + ": ");
    String entrada = leitor.nextLine();

    try {
        double preco = Double.parseDouble(entrada);
        totalCompra += preco;
        itensLidos++;
    } catch (NumberFormatException e) {
        System.out.println("Preço inválido! Tente novamente.");
    }
}

System.out.println("Total da compra: " + totalCompra);
```

DICA

Envolver a conversão de texto para número em um bloco try-catch evita que uma entrada inválida do usuário derrube o programa inteiro com uma exceção não tratada. É a forma correta de validar dados vindos do console.

07

O Debugger do IntelliJ

Inspecionando a execução do programa passo a passo

Vendo o código rodar, uma linha por vez

O debugger do IntelliJ permite acompanhar a execução do programa linha a linha, observando o valor real de cada variável naquele instante — uma ferramenta essencial para investigar bugs que não fazem sentido só de olhar o código parado. O primeiro passo é adicionar um breakpoint, clicando na margem esquerda da linha onde a execução deve pausar.

A partir de um breakpoint, três atalhos cobrem a maior parte do trabalho: F8 (Step Over) avança para a próxima linha sem entrar dentro de métodos chamados; F7 (Step Into) entra no método que está sendo chamado naquela linha, permitindo inspecionar sua execução interna; e F9 (Resume) retoma a execução normal até encontrar o próximo breakpoint.

DICA

Use o debugger sempre que um valor parecer errado e a causa não for óbvia lendo o código. Colocar um breakpoint pouco antes do trecho suspeito e avançar com F8 costuma revelar o momento exato em que o valor se desvia do esperado.

Fim de apostila

Com switch, laços, escopo e Scanner, seu programa já consegue interagir com o usuário, processar decisões e reagir a diferentes entradas.

Somado ao debugger do IntelliJ, esse é o conjunto de ferramentas que sustenta a construção de qualquer aplicação Java real.

JavaStudiesHub